Git Workflow Guide for the Team

1. General Git Rules for the Team

- Always pull the latest changes before starting work (git pull origin main).
- Use meaningful commit messages (e.g., 'Fixed login bug' instead of 'Fixed stuff').
- Do not commit large files directly; use Git LFS if needed.
- Resolve conflicts carefully and communicate with the team if needed.

2. Initial Setup (First Time Only)

- Clone the repository:
  git clone https://github.com/IngeRi92/adoption-platform.git
- Move into the project folder:
  cd adoption-platform
- Configure your Git user details:
  git config --global user.name 'Your Name'
  git config --global user.email 'your.email@example.com'

3. Daily Workflow

- Step 1: Pull the latest changes:
  git pull origin main
- Step 2: Create a new branch for your task:
  git checkout -b feature-branch-name
- Step 3: Make your changes and save files.
- Step 4: Check modified files:
  git status
- Step 5: Add your changes:
  git add .
- Step 6: Commit your work:
  git commit -m 'Short description of change'
- Step 7: Push the branch to GitHub:
  git push origin feature-branch-name

- Step 8: Create a Pull Request (PR) in GitHub and request review.
- Step 9: Once approved, merge it into main.

4. Handling Merge Conflicts

- If you get a merge conflict, Git will show which files have conflicts.
- Open the file in Visual Studio and manually edit the conflicting lines.
- After resolving, mark it as resolved:
  git add resolved-file
- Commit and push:
  git commit -m 'Resolved merge conflict'
  git push

5. Emergency Recovery

- If you committed something wrong, undo the last commit (without losing changes):
  git reset --soft HEAD~1
- If you completely messed up and want to reset your branch to the last known good state:
  git reset --hard origin/main

6. Who is Responsible for GitHub Maintenance?

- The DevOps & Full-Stack Support team member will oversee GitHub maintenance.
- Other team members should follow this guide to keep Git history clean and avoid conflicts.

Conclusion

By following these steps, our team can work efficiently and avoid Git-related problems.
If unsure, ask for help before pushing changes!